# Object Oriented Programming(C++)
# (CSE 1211)

## C++: Control Statement & Functions

**Md. Mahfuzur Rahman**
Lecturer,
**Prime University**

# C++ Control Statements are:

- if else
- switch
- for loop
- while loop
- do while loop
- break statement
- continue statement
- goto statement

# C++: if-else

In C++ programming, if statement is used to test the condition. There are various types of if statements in C++.

- if statement
- if-else statement
- nested if statement
- if-else-if ladder

# C++: if Statement

The C++ if statement tests the condition. It is executed if condition is true.

```
if(condition)
{
//code to be executed
}
```

## C++: if Example

```
#include <iostream>
using namespace std;
int main () {
int num = 10;
if (num % 2 == 0)
        {
            cout<<"It is even number";
        }
  return 0;
}
```

# C++: if Statement

```
if (20 > 18) {
  cout << "20 is greater than 18";
}
```

```
int x = 20;
int y = 18;
if (x > y) {
  cout << "x is greater than y";
}
```

# C++: if-else Statement

The C++ if-else statement also tests the condition. It executes if block if condition is true otherwise else block is executed.

```
if(condition)
{
//code if condition is true
}
else
{
//code if condition is false
}
```

## C++: if-else Example

```
#include <iostream>
using namespace std;
int main () {
  int num = 11;
        if (num % 2 == 0)
        {
           cout<<"It is even number";
        }
        else
        {
           cout<<"It is odd number";
        }
  return 0;
}
```

# C++: if-else-if ladder Statement

The C++ if-else-if ladder statement executes one condition from multiple statements.

```
if(condition1)
{
//code to be executed if condition1 is true
}
else if(condition2)
{
//code to be executed if condition2 is true
}
else if(condition3){
//code to be executed if condition3 is true
}
...
else{
//code to be executed if all the conditions are false
}
```

# C++: if-else-if ladder Statement

```
int time = 22;
if (time < 10) {
  cout << "Good morning.";
} else if (time < 20) {
  cout << "Good day.";
} else {
  cout << "Good evening.";
}
// Outputs "Good evening."
```

# C++: if else-if Example

```
#include <iostream>
using namespace std;
int main () {
    int num;
    cout<<"Enter a number to check grade:";
    cin>>num;
        if (num <0 || num >100)
        {
            cout<<"wrong number";
        }
        else if(num >= 0 && num < 40){
            cout<<"Fail";
        }
        else if (num >= 40 && num < 45)
        {
            cout<<"D Grade";
        }
        else if (num >= 45 && num < 50)
        {
            cout<<"C Grade";
        }
        else if (num >= 50 && num < 55)
        {
            cout<<“C+ Grade";
        }
        else if (num >= 55 && num < 60)
        {
            cout<<“B- Grade";
        }
        else if (num >= 60 && num <65)
        {
            cout<<“B Grade";
        }
}
```

# Short Hand If...Else (Ternary Operator)

*variable* = (*condition*) ? *expressionTrue* : *expressionFalse*;

```
int time = 20;
if (time < 18) {
  cout << "Good day.";
} else {
  cout << "Good evening.";
}
```

```
int time = 20;
string result = (time < 18) ? "Good day." : "Good evening.";
cout << result;
```

# Some Programs using if-else statement

- Write a c++ program to find a Positive or Negative Number using if-else statement.
- Write a c++ program to find an Odd and Even number using if-else statement.
- Write a c++ program to find large and small numbers using if-else statement.
- Write a c++ program to find pass or fail using if-else statement.
- Write a c++ program to find an absolute value using if-else statement.
- Write a c++ program to find a letter grade using if-else statement.
- Write a c++ program to find vowel and consonant using if-else statement.
- Write a c++ program to find leap year using if-else statement.

# C++: switch

```
switch(expression) {
  case x:
    // code block
    break;
  case y:
    // code block
    break;
  default:
    // code block
}
```

# C++: switch

The C++ switch statement executes one statement from multiple conditions. It is like if-else-if ladder statement in C++.

```
switch(expression){
case value1:
 //code to be executed;
 break;
case value2:
 //code to be executed;
 break;
.....

default:
 //code to be executed if all cases are not matched;
 break;
}
```

# C++ Switch Example

```
#include <iostream>
using namespace std;
int main () {
      int num;
      cout<<"Enter a number to check grade:";
      cin>>num;
         switch (num)
         {
            case 10: cout<<"It is 10"; break;
            case 20: cout<<"It is 20"; break;
            case 30: cout<<"It is 30"; break;
            default: cout<<"Not 10, 20 or 30"; break;
         }
   }
```

```cpp
int day = 4;
switch (day) {
  case 1:
    cout << "Monday";
    break;
  case 2:
    cout << "Tuesday";
    break;
  case 3:
    cout << "Wednesday";
    break;
  case 4:
    cout << "Thursday";
    break;
  case 5:
    cout << "Friday";
    break;
  case 6:
    cout << "Saturday";
    break;
  case 7:
    cout << "Sunday";
    break;
}
// Outputs "Thursday" (day 4)
```

# Some Programs using switch statement

- Write a c++ program to find a digit using switch statement.
- Write a c++ program to find vowel and consonant using switch statement.
- Write a c++ program to find a letter grade using switch statement.

# C++: for loop

The C++ for loop is used to iterate a part of the program several times. If the number of iteration is fixed, it is recommended to use for loop than while or do-while loops.

The C++ for loop is same as C/C#. We can initialize variable, check condition and increment/decrement value.

```
for(initialization; condition; incr/decr)
{
//code to be executed
}
```

## C++ For Loop Example

```
#include <iostream>
using namespace std;
int main() {
        for(int i=1;i<=10;i++){
          cout<<i <<"\n";
        }
    }
```

# C++: for loop

Using for loop

```
int sum = 0;
for (int i = 1; i <= 5; i++) {
  sum = sum + i;
}
cout << "Sum is " << sum;
```

# C++: Nested for loop

In C++, we can use for loop inside another for loop, it is known as nested for loop. The inner loop is executed fully when outer loop is executed one time. So if outer loop and inner loop are executed 4 times, inner loop will be executed 4 times for each outer loop i.e. total 16 times.

## C++ Nested For Loop Example

```
#include <iostream>
using namespace std;
int main () {
        for(int i=1;i<=3;i++){
            cout<<“when i is: “<<i<<endl;
            for(int j=1;j<=3;j++){
           cout<<i<<" "<<j<<"\n";
          }
        }
    }
```

```
#include <iostream>
using namespace std;

int main() {

   int rows;

   cout << "Enter number of rows: ";
   cin >> rows;

   for(int i = rows; i >= 1; --i) {
      for(int j = 1; j <= i; ++j) {
         cout << "* ";
         cout << j << " ";
      }
      cout << endl;
   }

   return 0;
}
```

# C++: infinite for loop

If we use double semicolon in for loop, it will be executed infinite times. Let's see a simple example of infinite for loop in C++.

```
#include <iostream>
using namespace std;

int main () {
        for (; ;)
          {
                cout<<"Infinitive For Loop";
          }
    }
```

# C++: while loop

In C++, while loop is used to iterate a part of the program several times. If the number of iteration is not fixed, it is recommended to use while loop than for loop.

```
while(condition)
{
//code to be executed
}
```

**C++ While Loop Example**

```
#include <iostream>
using namespace std;
int main() {
 int i=1;
        while(i<=10)
       {
           cout<<i <<"\n";
           i++;
         }
    }
```

**C++ While Loop Example**

```
#include <iostream>
using namespace std;
int main()
{
   char ch;
   cout<<"Press any Key to continue or Press Q
            or q to quit"<<endl;
   cin>>ch;
   while(ch!='Q'&&ch!='q')
   {
      cout<<"You didn't press q or Q"<<endl;
      cin>>ch;
   }
   cout<<“You pressed Q or q"<<endl;
}
```

# C++: Nested while loop

In C++, we can use while loop inside another while loop, it is known as nested while loop. The nested while loop is executed fully when outer loop is executed once.

**C++ Nested While Loop Example**

```
#include <iostream>
using namespace std;
int main () {
        int i=1;
          while(i<=3)
          {
             int j = 1;
             while (j <= 3)
           {
           cout<<i<<" "<<j<<"\n";
           j++;
          }
          i++;
       }
   }
```

# C++: Infinitive while loop Example

We can also create infinite while loop by passing true as the test condition.

```
#include <iostream>
using namespace std;
int main () {
        while(true)
          {
                cout<<"Infinitive While Loop";
          }
    }
```

# C++: do-while loop

The C++ do-while loop is used to iterate a part of the program several times. If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use do-while loop.

The C++ do-while loop is executed at least once because condition is checked after loop body.

```
do
{
//code to be executed
}
while(condition);
```

## C++ do-while Loop Example

```
#include <iostream>
using namespace std;
int main() {
    int i = 1;
        do{
            cout<<i<<"\n";
            i++;
        } while (i <= 10) ;
}
```

# C++: Nested do-while Loop

In C++, if you use do-while loop inside another do-while loop, it is known as nested do-while loop. The nested do-while loop is executed fully for each outer do-while loop.

```
#include <iostream>
using namespace std;
int main() {
    int i = 1;
        do{
            int j = 1;
            do{
              cout<<i<<"\n";
                j++;
            } while (j <= 3) ;
            i++;
         } while (i <= 3) ;
}
```

# C++ Infinitive do-while Loop

In C++, if you pass **true** in the do-while loop, it will be infinitive do-while loop.

```
do
{
//code to be executed
}
while(true);
```

## C++ Infinitive do-while Loop Example

```
#include <iostream>
using namespace std;
int main() {
      do{
           cout<<"Infinitive do-while Loop";
          }
       while(true);
}
```

# C++: break statement

The C++ break is used to break loop or switch statement. It breaks the current flow of the program at the given condition. In case of inner loop, it breaks only inner loop.

jump-statement;
**break**;

## C++: break statement Example

```
#include <iostream>
using namespace std;
int main() {
    for (int i = 1; i <= 10; i++)
        {
            if (i == 5)
            {
                break;
            }
        cout<<i<<"\n";
        }
}
```

# C++: break statement with Inner Loop

The C++ break statement breaks inner loop only if you use break statement inside the inner loop.

**Let's see the example code:**

```
#include <iostream>
using namespace std;
int main()
{
   for(int i=1;i<=3;i++){
        for(int j=1;j<=3;j++){
            if(i==2&&j==2){
                break;
                }
                cout<<i<<" "<<j<<"\n";
                }
        }
}
```

# C++: continue statement

The C++ continue statement is used to continue loop. It continues the current flow of the program and skips the remaining code at specified condition. In case of inner loop, it continues only inner loop.

jump-statement;
**continue**;

## C++: continue statement Example

```
#include <iostream>
using namespace std;
int main()
{
    for(int i=1;i<=10;i++){
        if(i==5){
            continue;
        }
        cout<<i<<"\n";
    }
}
```

# C++: continue statement with Inner Loop

C++ Continue Statement continues inner loop only if you use continue statement inside the inner loop.

```
#include <iostream>
using namespace std;
int main()
{
 for(int i=1;i<=3;i++){
          for(int j=1;j<=3;j++){
           if(i==2&&j==2){
              continue;
                    }
              cout<<i<<" "<<j<<"\n";
                 }
          }
}
```

# C++: goto statement

The C++ goto statement is also known as jump statement. It is used to transfer control to the other part of the program. It unconditionally jumps to the specified label.
It can be used to transfer control from deeply nested loop or switch case label.

## C++: goto statement Example

```
#include <iostream>
using namespace std;
int main()
{
ineligible:
        cout<<"You are not eligible to vote!\n";
    cout<<"Enter your age:\n";
    int age;
    cin>>age;
    if (age < 18){
            goto ineligible;
    }
    else
    {
     cout<<"You are eligible to vote!";
    }
}
```